Design and Implementation of a ResNet-Style Convolutional Neural Network for CIFAR-10 Image Classification

KIIT University

Submitted By:

Name: Aayush Sah

Roll: 23053488

Section: CSE-48

Submitted to:

Rinku Datta Rakshit

Associate Professor

School of Computer Engineering,

KIIT University

TABLE OF CONTENTS

1. Abstract

This report presents the design, development, and evaluation of a Convolutional Neural Network (CNN) for image classification on the CIFAR-10 dataset, following a structured and iterative improvement approach. The study begins with a baseline CNN model and progressively enhances its performance through systematic modifications, including data augmentation, batch normalization, dropout regularization, and optimization of training strategies.

A key contribution of this work is the implementation of a Residual Network (ResNet)-style architecture to address the degradation and vanishing gradient problems commonly observed in deep neural networks. By incorporating skip connections, the model enables efficient gradient flow and facilitates deeper network training without loss of performance.

The iterative optimization process resulted in a significant improvement in classification accuracy, increasing from an initial 71.21% to a final accuracy of 90.08%. This demonstrates the effectiveness of residual learning and modern architectural design principles in improving model generalization and stability.

To validate its practical applicability, the final trained model was deployed as a full-stack web application using FastAPI, allowing real-time image classification through a user-friendly interface. This end-to-end pipeline highlights not only model performance but also its usability in real-world scenarios.

2. Introduction and Dataset Preprocessing

Reference File: 0-preprocessing.ipynb

2.1 Overview of CIFAR-10 Dataset:

The **CIFAR-10** dataset consists of 60,000 color images of size 32×32 pixels, categorized into 10 distinct classes. The dataset is divided into 50,000 training samples and 10,000 testing samples.

The following preprocessing steps were applied:

2.2Normalization:
Pixel intensity values were scaled from the range [0, 255] to [0, 1] to ensure numerical stability and faster convergence during training.

2.3 Label Encoding:
Class labels were transformed into one-hot encoded vectors to make them compatible with the categorical cross-entropy loss function used for multi-class classification.

3. ResNet Architecture Implementation

Reference File: fork-of-resnet-style-model.ipynb

3.1 Motivation: Degradation Problem

As convolutional neural networks become deeper, their performance often saturates and then degrades, not due to overfitting but because of optimization difficulties. This phenomenon is known as the **degradation problem**, where adding more layers leads to higher training error.

One of the primary causes of this issue is the **vanishing gradient problem**, where gradients become extremely small during backpropagation, preventing early layers from learning effectively. As a result, deeper networks fail to converge properly and do not achieve better accuracy despite increased capacity.

To address this limitation, Residual Networks (ResNet) introduce a novel architectural concept known as **residual learning**, which enables efficient training of deep neural networks.

3.2 Residual Learning Concept

Instead of learning a direct mapping from input (x) to output (H(x)), ResNet reformulates the problem to learn a residual function: [F(x) = H(x) - x]

Thus, the original function becomes: [H(x) = F(x) + x]

This is implemented using **skip (shortcut) connections**, where the input is directly added to the output of a few stacked layers.

This approach provides the following advantages:

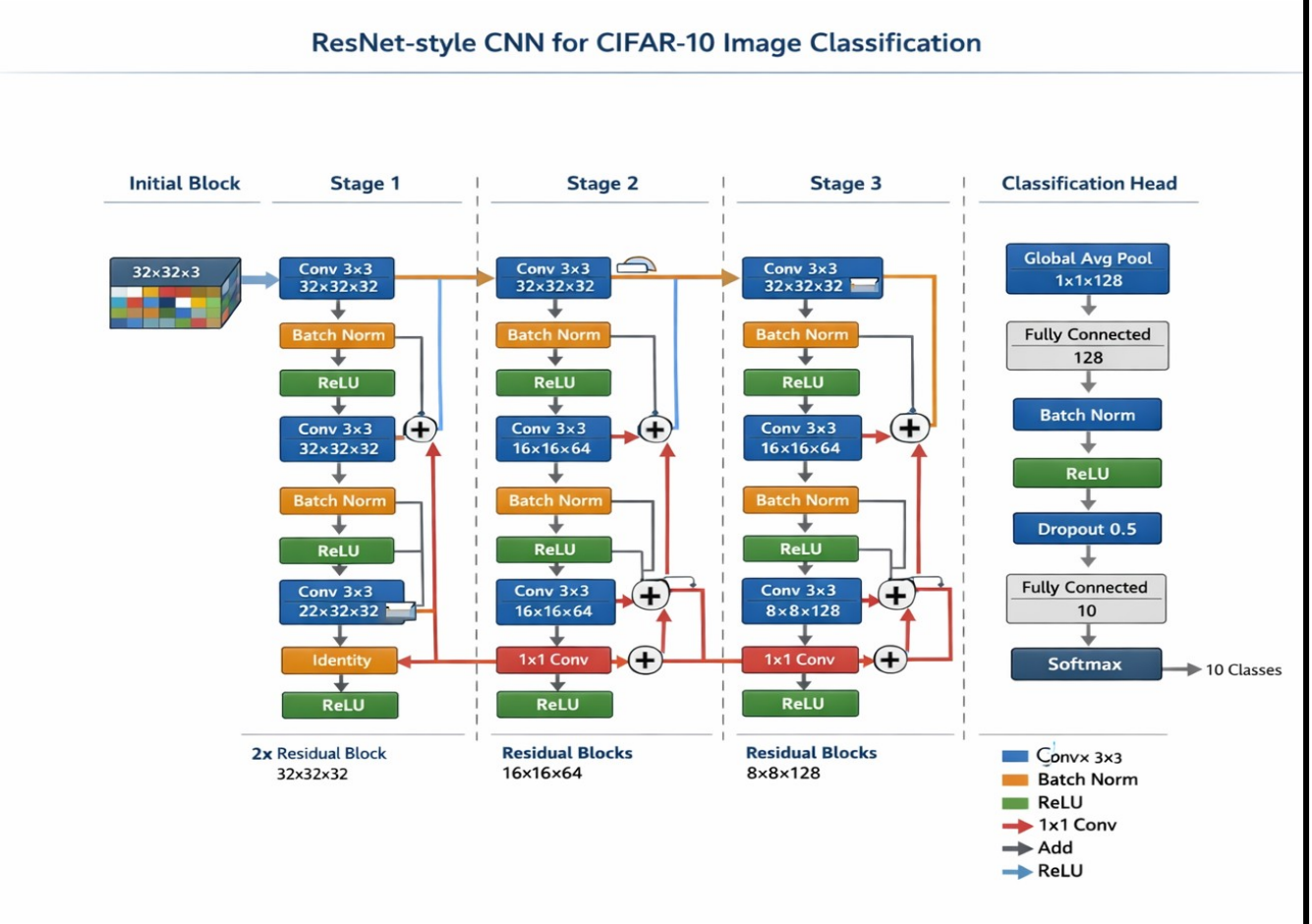
- Enables smooth gradient flow across layers
- Prevents vanishing gradient issues
- Allows deeper networks to train effectively
- Improves convergence speed and accuracy

3.3 Architecture Design

The implemented model follows a ResNet-style architecture tailored for CIFAR-10 classification, with an input size of 32×32×3. The network is composed of an initial convolutional block followed by multiple residual stages and a classification head.

Initial Block:

- Conv2D (32 filters, 3×3 kernel, stride 1, padding='same')
- Batch Normalization
- ReLU Activation



Residual Block Structure:

Each residual block consists of:

- Conv2D → BatchNorm → ReLU
- Conv2D → BatchNorm
- Shortcut connection (identity or 1×1 convolution)
- Element-wise addition
- ReLU activation

CODE:

```
def residual_block(x, filters, downsample=False
        shortcut = x
        stride = 2 if downsample else 1
        x = layers.Conv2D(filters, (3,3), strides=stride, padding='same')(x)
        x = layers.BatchNormalization()(x)
        x = layers.Activation('relu')(x)
        x = layers.Conv2D(filters, (3,3), padding='same')(x)
        x = layers.BatchNormalization()(x)

if downsample or shortcut.shape[-1] != filters:
    shortcut = layers.Conv2D(filters, (1,1), strides=stride, padding='same')(shortcut)
```

```
    shortcut = layers.BatchNormalization()(shortcut)
x = layers.Add()([x, shortcut])
 x = layers.Activation('relu')(x)
return x
```

Stage-wise Architecture:

- **Stage 1:**
  - Two residual blocks with 32 filters
  - No spatial downsampling
  - Output size: 32×32×32
- **Stage 2:**
  - First block uses stride=2 for downsampling
  - Two residual blocks with 64 filters
  - Output size: 16×16×64
- **Stage 3:**
  - First block uses stride=2 for downsampling
  - Two residual blocks with 128 filters
  - Output size: 8×8×128

Classification Head:

- Global Average Pooling
- Dense layer (128 units)
- Batch Normalization
- ReLU Activation
- Dropout (0.5)
- Dense layer (10 units, Softmax)

CODE:

```
inputs = layers.Input(shape=(32, 32, 3))
x = layers.Conv2D(32, (3,3), padding='same')(inputs)
x = layers.BatchNormalization()(x)
x = layers.Activation('relu')(x)
x = residual_block(x, 32)
x = residual_block(x, 32)
x = residual_block(x, 64, downsample=True)
x = residual_block(x, 64)
x = residual_block(x, 128, downsample=True)
x = residual_block(x, 128)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(128)(x)
x = layers.BatchNormalization()(x)
x = layers.Activation('relu')(x)

x = layers.Dropout(0.5)(x)
outputs = layers.Dense(10, activation='softmax')(x)
model = models.Model(inputs, outputs)
```

This design ensures efficient feature extraction while maintaining a manageable number of

parameters.

3.4 Skip Connections and Their Role

Skip connections are the core component of the ResNet architecture. They allow the input of a layer to bypass intermediate transformations and be directly added to the output.

There are two types of shortcut connections used:

- **Identity Shortcut:** Used when input and output dimensions are the same
- **Projection Shortcut (1×1 Convolution):** Used when dimensions differ (e.g., during downsampling)

The key benefits of skip connections include:

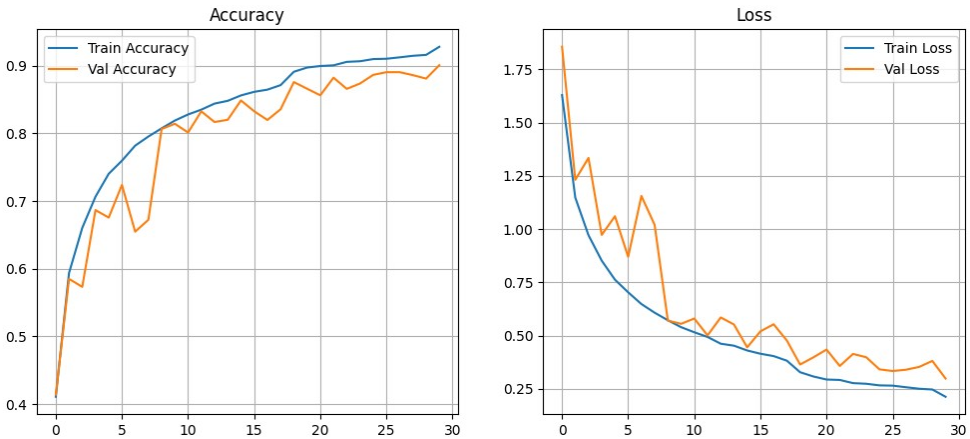
- Improved gradient propagation during backpropagation
- Prevention of performance degradation in deeper networks
- Better feature reuse across layers
- Stabilized training process

By enabling uninterrupted information flow, skip connections allow the network to learn deeper representations without suffering from optimization issues, which significantly contributes to the final model accuracy of **90.08%**.

4. Experimental Results and Analysis

4.1 Training and Validation Performance

The model was trained on the CIFAR-10 dataset using a ResNet-style architecture. During training, both training and validation accuracy showed consistent improvement, indicating stable learning behavior.



Regularization techniques such as Batch Normalization and Dropout helped reduce overfitting, while the use of Global Average Pooling minimized parameter complexity. The incorporation of residual connections ensured smooth gradient flow, allowing the network to train effectively without degradation.

4.2 Accuracy Evaluation

The final model achieved a test accuracy of 90.08%, representing a significant improvement over the baseline model accuracy of 71.21%. This corresponds to an overall increase of 18.87 percentage points.

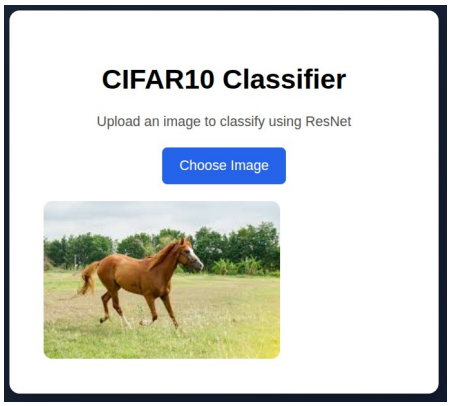
The performance gain can be attributed to:

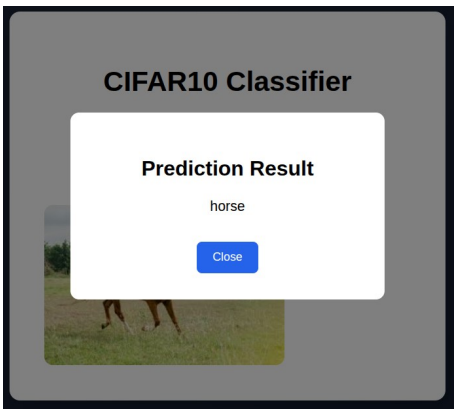
- Use of residual learning (skip connections)
- Improved network depth without degradation
- Effective regularization techniques
- Optimized training strategies

These results demonstrate that the ResNet-style architecture significantly enhances model generalization and classification performance on the CIFAR-10 dataset.

5. Deployment: FastAPI Web Application

The trained ResNet model was deployed as a web application using FastAPI to enable real-time image classification.





5.1 Backend Implementation:

The backend was developed using FastAPI, where the trained model was loaded using Keras. It handles image input, performs prediction, and returns the corresponding CIFAR-10 class label.

5.2 Inference Pipeline:

Input images are resized to 32×32, normalized to the range [0, 1], and processed by the model to generate predictions. The output is mapped to the respective class label.

5.3 Frontend Interface:

A simple HTML/CSS interface allows users to upload images and view predictions instantly. The frontend communicates with the backend API to display results dynamically.

6. Conclusion

This project demonstrated a structured approach to developing an effective image

classification model using Convolutional Neural Networks on the CIFAR-10 dataset. Starting from a basic CNN, the model was progressively improved through the application of modern deep learning techniques, culminating in the implementation of a ResNet-style architecture.

The introduction of residual learning and skip connections successfully addressed the degradation and vanishing gradient problems, enabling deeper network training and significantly improving performance. The final model achieved an **accuracy of 90.08%**, representing a substantial improvement over the initial baseline.

In addition to model development, the project emphasized practical usability by deploying the trained model as a FastAPI-based web application. This demonstrated the feasibility of integrating deep learning models into real-world systems for interactive use.

Overall, the project highlights the importance of architectural design, optimization strategies, and deployment in building robust and scalable machine learning solutions.

7. References

[1] V. Kant and K. S. Kumar, "Automated Flower Recognition using ResNet," OTCON, 2025.

[2] E. D and N. P. G. Bhavani, "ResNet-Based Satellite Image Classification," ICOSEC, 2023.

[3] A. M et al., "Lung Cancer Detection using ResNet-50," ICAIT, 2024.

[4] U. Archana et al., "Plant Disease Detection using ResNet," ICICT, 2023.

[5] R. Pandya et al., "ResNet Features for Parkinson's Disease Diagnosis," InCACCT, 2023.